

复旦大学计算机科学技术学院
2016~2017 学年第 一 学期期末考试试卷
☒A 卷 ☐B 卷

课程名称: 操作系统 课程代码: COMP130110.01-04

开课院系: 计算机科学技术学院 考试形式: 闭卷

姓 名: _____ 学 号: _____ 专 业: _____

题 号	1	2	3	4	5	6	7	8	总 分
得 分									

(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

NO aids (books, notes, discussion etc.) are allowed. Ask the instructor if you have any question.

1. Choice (13 * 3% = 39%).

- 1) A computer system consists of multiple layers, say (1) Computer hardware, (2) Utilities, (3) Application programs, and (4) Operating system. The correct hierarchical configuration (bottom up) should be: _____

A. 1/2/3/4 B. 1/2/4/3
C. 1/4/2/3 D. 1/4/3/2

Answer: C

- 2) Which of the following are shared by all of the threads in a process? Assume a kernel-level thread implementation. _____

A. Register value
B. File descriptors
C. Scheduler priority
D. Thread local variables

Answer: B

- 3) The purpose of a TLB is _____
 A. To cache page translation information
 B. To cache frequently used pages
 C. To hold register values while a process is waiting to be run
 D. To hold the start the length of the page table
Answer: A
- 4) Which of the following is NOT included in an i-node in UNIX-like systems? ____

 A. File owner
 B. File name
 C. File modification time
 D. Pointer to the first data block
Answer: B
- 5) Which layer of I/O software issues a sequence of commands to the device controller? _____
 A. User-space I/O software
 B. Device Independent I/O software
 C. Device driver
 D. Interrupt handler
Answer: C
- 6) Suppose a system uses compaction method to solve the problem of external fragmentation for dynamic partitioning. When should the physical addresses be generated? _____
 A. at compile time B. at linking time
 C. at load time D. at run time
Answer: D
- 7) For a given process, the length of its page reference stream is 20, which consists of 8 distinct pages. If demand paging is used and 4 frames (initially all empty) are allocated to this process, what is the lower bound for the number of page faults during the process' execution? _____
 A. 4 B. 8
 C. 12 D. 20
Answer: B
- 8) Having large block size in file system will result in _____
 A. Fast data access due to fewer disk seeks for the same amount of data read.
 B. Less waste of disk space due to internal fragmentation.
 C. More information needed to keep track of blocks allocated to a file.
 D. Less waste of disk space due to external fragmentation.
Answer: A
- 9) Which event can cause CPU preemption? _____
 A. A time quantum exhausts.

- B. A process issues an I/O request.
- C. A process exits.
- D. All of the above.

Answer: A

- 10) A typical pattern for information processing is input, processing, and output. Suppose that the three tasks are designed as separate processes, the case can be formulated as a _____.
- A. Producer/consumer problem
 - B. Reader/Writer problem
 - C. Dining philosophers problem
 - D. Barber problem

Answer: A

- 11) A semaphore s is initialized as 2. After four processes (P1, P2, P3, and P4) have done the following operations sequentially:
 $(P1, \text{wait}(s)) \rightarrow (P2, \text{wait}(s)) \rightarrow (P3, \text{wait}(s)) \rightarrow (P4, \text{signal}(s))$.
 Which process is still in BLOCKED state? _____
- A. none
 - B. P1
 - C. P2
 - D. P3

Answer: A

- 12) A UNIX file system has 4-KB blocks and 4-byte disk addresses. The i-node of file 'X' requires 8 direct entries and an indirect block which is half full. How large is the file in blocks? _____
- A. 16
 - B. 264
 - C. 520
 - D. 1032

Answer: C

- 13) In a block-oriented I/O system, let T denote the time of input transfers made to buffer, M the time of moving a block to user space when needed, and C the time of data processing. If we use double buffering strategy, and it holds that $C > T$, the average execution time per block should be _____
- A. $T+C$
 - B. $T+M+C$
 - C. $T+M$
 - D. $C+M$

Answer: D

2. Short Answer (29%).

- (1) A paging system uses 16-bit address and 4KB-sized pages. The following shows the page tables of two running processes, Process 1 and Process 2. Translate the logical addresses in the table below to their corresponding physical addresses, and fill the table entries with your answer. (6 points)

	Process 1		Process 2
0	3	0	1
1	0	1	6
2	7	2	4
3	5	3	2

<i>Process</i>	<i>Logical address</i>	<i>Page #</i>	<i>Offset in Page</i>	<i>Physical Address</i>
Process 1	12,000			
Process 2	8,000			

Answer: 2 3808 32480
1 3904 28480

- (2) Please compare and contrast the applicability of the Peterson's algorithm to fulfil the mutual exclusion with threads and processes. (Hints: Please consider the shared variable, e.g. turn) (6 points)

Referential answer: The Peterson's algorithm need agents to use it can access a shared variable. We have no difficulty to use it with multiple threads of a process, since all the threads operate in a common address space. (3 points)

But in multiple processes situation, the implementation is much difficult unless these processes share something, e.g. sharable address space, or file, or certain kernel buffer with corresponding system call. Since in typical situations, different processes (including their address space) are isolated by operating systems. (3 points)

- (3) Consider a computer with spinlock as the mechanism of mutual exclusion. There are two processes, H, with high priority, and L, with low priority. The scheduling rules are such that H is run whenever it is in ready state. Please construct a scenario of the priority inversion on a uniprocessor computer. Does it definitely be the same in a SMP (Symmetrical Multi-Processing) environment? (6 points) (Note: A spin lock refers to a technique in which a process is busy-waiting until it gets permission to enter its critical section)

Referential answer: On a uniprocessor computer, at a certain moment, with L in its critical region, H becomes ready to run H now begins busy waiting, but since L is never scheduled while H is running, L never gets the chance to leave its critical region, so H loops forever. (3 points)

In a SMP environment, the situation might be changed. H might be executed on the other processors. So, it is not definite that the priority inversion will happen. (3 points)

points)

- (4) Three resources are used concurrently. To guarantee mutual exclusion the access to the resources is protected by semaphores named LA, LB, and LC as shown in the code below for three threads

Thread 1	Thread 2	Thread 3
<code>wait(LA);</code> <code>//use resource A</code> <code>signal(LA);</code> <code>wait (LB);</code> <code>//use resource B</code> <code>signal(LB);</code>	<code>wait(LC);</code> <code>//use resource C</code> <code>signal(LC);</code> <code>wait(LB);</code> <code>//use resource B</code> <code>signal(LB);</code>	<code>wait(LA);</code> <code>wait(LB)</code> <code>wait(LC)</code> <code>//use resources A,B, and C</code> <code>signal(LA);</code> <code>signal(LB);</code> <code>signal(LC);</code>

Is it possible for this code to result in deadlock? If so, explain how. If not, explain what prevents deadlock from occurring. (6 points)

Answer: No, deadlock cannot occur in this example. There can be no circular wait among these threads. Threads 1 and 2 only contend for ownership of one lock (LB) and cannot form a cycle. Threads 1 and 3 contend for two locks but acquire their locks in the same order and cannot form a cycle. Threads 2 and 3 contend over two locks and in different orders. However, if thread 2 is holding lock LC (blocking thread 3) it will release this lock before acquiring lock LB. In this case either thread 2 will acquire lock LB or thread 3 having acquired lock LB will be able to acquire lock LB.

- (5) An interactive system uses round robin (or time-slicing) scheduling. It tries to give guaranteed response to requests as better as possible. A designer claims that it can be done easily by cutting the time quantum (or slice) further when needed. Please give your comments on this design, and give your design guideline as well. (5 points)

Reference point:

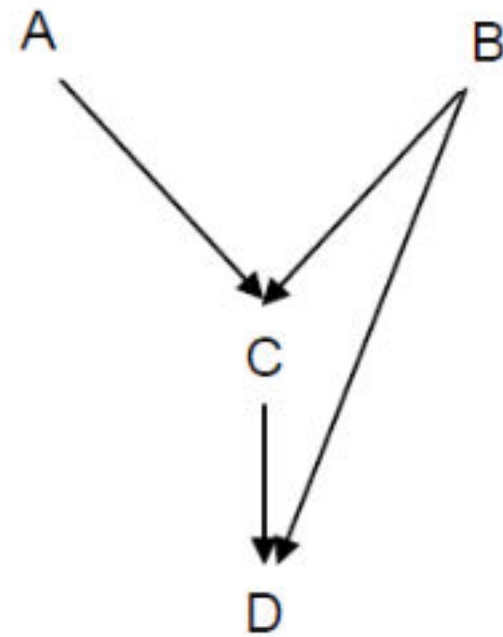
With round robin, the principal design issue is the length of the time quantum. Generally, if the quantum gets short, the response can be improved. (1%)

Due to the overhead, the process has limit. When the quantum becomes too small, processor utilization decreases, and at a certain point, users will then experience a sudden increase of response times because their requests must pass through the round-robin queue several times. (2%)

One useful guide is the time quantum should be slightly greater than the time required for a typical interaction. (2%)

3. Analysis (32%).

- (1) There are 4 tasks, named A, B, C, and D. They must obey the constraints as shown in the graph below. An edge in the graph depicts the happened-before relation between two tasks. For example, activity C cannot be performed unless both activity A and activity B have been completed. Please use semaphore(s) to complete the program pseudo-code below to enforce these constraints. (10 points)



```
// declare any semaphores you need here;
// be sure to show how to initialize them
```

```
A()
{
```

```
    Perform activity A;
```

```
}
```

```
B()
{
```

```
    Perform activity B;
```

```
}
```

```
C()
{
```

```
    Perform activity C;
```

```
}
```

```
D()
{
```

```
    Perform activity D;
```

```
}
```

```
int main()
{
    Parbegin(A, B, C, D);
}
```

Answer:

// declare any semaphores you need here; // be sure to show how to initialize them

semaphore s_ac = 0, s_bc = 0, s_cd = 0, s_bd = 0; (2 points)

```
A() {  
    Perform activity A;  
    signal(s_ac); //0.5 point  
}
```

```
B() {  
    Perform activity B;  
    signal(s_bc); //1 point  
    signal(s_bd); //1 point  
}
```

```
C() {  
    wait(s_ac); //1 points  
    wait(s_bc); //1 points  
    Perform activity C;  
    signal(s_cd); //0.5 points  
}
```

```
static void * thread_D(void *_) {  
    wait(s_bd); //0.5 points  
    wait(s_cd); //0.5 points  
    Perform activity D;  
}
```

```
int main()  
{  
    Parbegin(A, B, C, D);  
}
```

- (2) Consider a virtual memory system whose virtual (linear) addresses are of 32 bits. The high-order 10 bits index into the page directory, the next 10 bits index into the page table, and the low-order 12 bits index into the page.
- What is the size of a page, in bytes, in the system? How many pages (at most) can a process have? (2 points)
 - If we make the following change: the high-order 8 bits index into the page directory, the next 8 bits index into the page table, and the low-order 16 bits index into the page. What are the advantages and disadvantages of the change? (4 points)
 - If we change virtual (linear) address such that: the low-order 10 bits index into the page directory, the next 10 bits index into the page table, and the high-order 12

bits index into the page. Will this work? If so, is it a good or bad idea? Justify your answer. (4 points)

Answer:

a) page size = 2^{12} ; max number of pages in a process: 2^{20}

b) Pros: less overhead due to paging structures and swapping to/from disk; more of virtual memory fits into the TLB at once

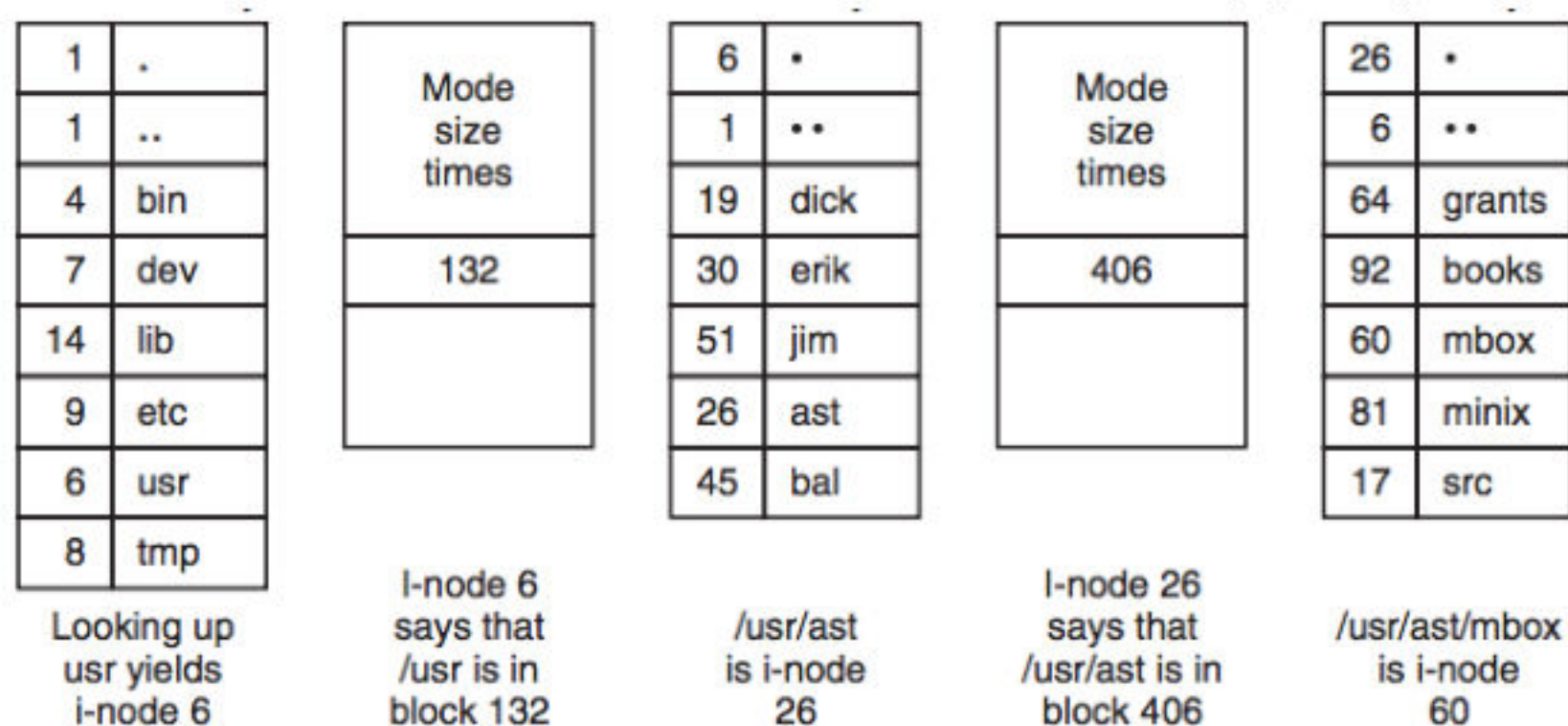
Cons: more wasted space due to internal fragmentation; less granularity for swapping; swapping an individual page is more expensive.

c) It will work, since there is still a valid mapping between virtual and physical addresses. It is a bad idea, though, because it eliminates the benefits of spatial locality. Neighboring addresses will translate to different pages, leading to high overhead in paging data structure and frequent swapping.

However, if the locality is not violated by other design consideration, the scheme is also OK.

(3) When a file is opened, the file system must take the file name supplied and locate its disk blocks. Let us consider how the file `/usr/ast/mbox` is looked up in a UNIX system using i-nodes. Please answer following questions.

a) Given a sketch of the system as below and assume the root directory is located in the block 1, the file is smaller than a block, explain how many disk accesses have to be executed if it reads the content of the file `/usr/ast/mbox`? (4 points)



b) It is known that parsing the pathname is necessary and expensive. Can you provide certain optimizing approach to speed up the work? Someone provides a mechanism that could maintain some information as below in memory as the system parses path names. Please recalculate the optimal number of disk accesses when there is this information. (4 points)

Path	I-node number
/usr	6
/usr/ast	26
/usr/ast/books	92
/usr/bal	45

c) Is there any potential problem with the mechanism in b) as a file might be deleted and created again? Explain your consideration and provide a solution to it? (4 points)

Answer:

a) Referential answer: 8 times, they are:

- Read the i-node for the / (i-node 1) [SEP]
- Read the / (block 1)
- Read the i-node for /usr (i-node 6) [SEP]
- Read the /usr directory (block 132) [SEP]
- Read the i-node for /usr/ast (i-node 26) [SEP]
- Read the /usr/ast directory (block 406) [SEP]
- Read the i-node for /usr/ast/mbox (i-node 60) [SEP]
- Read the /usr/ast/mbox directory (block somewhere) [SEP]

The three points should be assigned to the correct number of disk accesses, the alternative access pattern of i-node and corresponding block, and the result sequence.

b) Referential answer: it is reduced to 4, they are:

- Read the i-node for /usr/ast (i-node 26) [SEP]
- Read the /usr/ast directory (block 406) [SEP]
- Read the i-node for /usr/ast/mbox (i-node 60) [SEP]
- Read the /usr/ast/mbox directory (block somewhere) [SEP]

The three points should be assigned to the correct number of disk accesses, the cache mechanism it uses (using the longest substring matching), and the explanation about the principle of locality.

c) Referential answer: A problem with caching paths is that the mapping between file name and i-node number is not fixed as a file, e.g. /usr/ast/mbox is removed from the system and its i-node is reused for a different file owned by a different user. The cache entry will now be wrong and subsequent lookups will return the wrong i-node number. For this reason, when a file or directory is deleted, its cache entry and (if it is a directory) all the entries below it must be purged from the cache. The two points should be assigned to the reason and the solution, respectively.

